

FlexBook: Engineering Assignment

Welcome

Thanks for taking the time to work on this assignment. We've designed it to reflect the kind of problems we solve daily - multi-tenant systems with configurable business logic and realworld ambiguity.

We care about how you think AND what you build. We expect a fully working implementation backed by a well-reasoned architecture document.

The Problem

You are building the backend for FlexBook - a platform where different businesses (tenants) manage bookings for their resources. Each tenant has different booking rules, and these rules are not hardcoded. Tenants configure their own rules by composing primitives that you design.

The core challenge: Design a rule composition engine that is extensible - adding a new rule type should require only writing a new rule implementation, without modifying the engine itself.



Build this as if it were a real and robust product going to production. Think about edge cases, failure modes, and the decisions a senior engineer would need to make when the spec is silent. Every assumption, trade-off, and design choice you make along the way should be captured clearly in your **DECISIONS.md**.

Rule Primitives

Your platform must support the following rule types. Tenants compose these to define their booking behavior.

You must implement at least one rule from each category, plus the rule composition engine that makes them work together. Implement as many of the remaining rules as you can - we expect a thorough implementation, but depth

over breadth: a well-tested engine with 6-7 solid rules beats a fragile engine with all 12 .

Constraint Rules

These restrict when and how bookings can happen.

#	Rule	Description	Example
1	Time Boundary	Resource available only during specific windows	Mon-Fri 9am-6pm
2	Max Duration	Cap on booking length	Max 4 hours per session
3	Buffer Time	Enforce gaps between consecutive bookings	15 min cleanup between bookings
4	Capacity Limit	Max concurrent bookings on a resource	Only 3 trainers available at once
5	Cooldown	User can't rebook same resource within X hours	Can't rebook same room within 24 hours

#	Rule	Description	Example
6	Blackout Period	No bookings during specific date ranges	Dec 25 - Jan 3, no bookings allowed

Conditional Rules

These apply only when specific conditions are met.

#	Rule	Description	Example
7	Approval Required	Booking is created in PENDING status when a condition is true. You do not need to build the approval flow - just the status transition and the rule evaluation.	Group size > 10 triggers pending status
8	Dependent Booking	Resource X requires resource Y to also be booked	Tennis court requires a trainer
9	User Quota	User can't exceed N bookings per time period	Max 2 appointments per week per patient

Modification Rules

These alter booking behavior rather than blocking it.

#	Rule	Description	Example
10	Peak Pricing	Apply price multiplier during specific windows. Resources have a configurable base price per hour.	Base price 10,000/hr, 2x multiplier on weekday evenings 5pm-9pm
11	AutoConfirm	Automatically confirm booking (skip PENDING) if all conditions pass	Auto-confirm if group < 5 and within business hours

Bonus: Recurring Slot

This is significantly more complex than other rules and is optional.

#	Rule	Description	Example
12	Recurring Slot	A booking creation mode - a single API call that produces multiple individual bookings, each validated independently against all applicable rules. Must handle partial success (some instances may pass while others fail).	Book every Monday 5pm-6pm for 4 weeks

Rule Composition

Tenants combine rules using these mechanisms:

- AND - all rules must pass for a booking to be valid
- Priority / Override - higher priority rule wins on conflict (e.g., a blackout period overrides auto-confirm)
- Scoping - rules can apply at three levels:
 - Resource-specific - applies to one resource (e.g., Meeting Room A)
 - Resource type - applies to all resources of a type (e.g., all meeting rooms)
 - Tenant-wide - applies across all resources for the tenant

API Contract

Your system must expose these endpoints. Request/response shapes are yours to design.

```
POST /tenants # Create a tenant
POST /tenants/{id}/resources # Create a resource for a tenant
POST /tenants/{id}/rules # Create/compose rules for a tenant
POST /tenants/{id}/bookings # Create a booking (validated against rules)
POST /tenants/{id}/bookings/batch # Batch create bookings with per-item validation report
GET /tenants/{id}/bookings # Query bookings with filtering
```

Deliverables

1. Architecture Document - ARCHITECTURE.md
2. Fully Working Backend
3. Tests
4. Decision Log - DECISIONS.md

Important Notes

AI tools are welcome. Use whatever tools help you work effectively - AI assistants, code generators, documentation tools. We care about how effectively you use them and whether you understand what they produce. Be honest about your AI usage in your decision log.

Take your time. There is no fixed deadline - take the time you need to deliver your best work. Coordinate with your recruiter on a submission date that works for you.

Choose any tech stack from: Kotlin / Go / Rust

Submission

To submit your implementation, please follow these steps:

1. **Create a Private Repository** on GitHub.
2. **Develop your solution** in a branch and open a **Pull Request (PR)**.
3. **Add the following users** as collaborators and reviewers on the PR:
 - `RaviDhing04`
 - `itohiro73`
 - `praveen0raj`

Good luck - we're looking forward to seeing how you think and build.